

Acceso al repositorio original y a la base de datos

Informe de viabilidad · Intranet Black Moose Group (boilerplate) · 9 de septiembre de 2026

La conclusión, primero. Hoy el boilerplate no es una aplicación: es una réplica visual alimentada con datos escritos a mano a partir de capturas de pantalla. Funciona porque la app de producción está encendida y podemos mirarla. **Si mañana el live dejara de estar accesible, nuestra app seguiría arrancando, pero se quedaría congelada: sin forma de terminar el 29 % que falta, sin forma de verificar lo ya hecho y sin ningún camino corto hacia datos reales.** Recuperar esa capacidad no se mide en días: se mide en meses de ingeniería inversa sobre un sistema que ya existe y que podríamos, simplemente, leer.

1. Cómo estamos construyendo hoy

El método es el *calco por captura*: entramos en la app de producción con un usuario de prueba, fotografiamos cada pantalla, volcamos su DOM, medimos los colores con dos métodos independientes y reconstruimos el resultado en React. Los datos que se ven en nuestra copia son literales tecleados desde esas capturas.

Ha funcionado bien y ha producido mucho en dos meses. Estos son los números reales del repositorio a día de hoy:

37.315 LÍNEAS DE CÓDIGO	1.019 TESTS EN VERDE	472 COMMITTS EN 2 MESES	99 MB CAPTURAS DEL LIVE
-----------------------------------	--------------------------------	-----------------------------------	-----------------------------------

Y este es el precio que estamos pagando por ese resultado, también en números reales:

CONCEPTO	HOY	QUÉ SIGNIFICA
Líneas de datos escritas a mano	7.966	En 97 ficheros de <i>seed</i> . Ninguna procede de una base de datos: todas se han tecleado desde una captura.
Capturas de pantalla archivadas	413	Son nuestra única especificación. Si el live cambia, quedan obsoletas sin avisar.
Barridos completos del live	5	Desde julio. Cada uno cuesta una jornada de agente y hay que repetirlo porque el live se mueve.
Módulos conectados a un repositorio de datos	5 de 16	Los otros 11 leen literales importados directamente en el componente.
Métodos del adaptador de Supabase implementados	0 de 17	Los 17 lanzan <code>not implemented</code> . La arquitectura está preparada; no hay nada al otro lado.

2. El problema de fondo: la app crece más rápido de lo que la copiamos

El live no es un objetivo quieto. Es un sistema en producción, en uso diario, que crece por su cuenta:

MEDICIÓN	VALOR
Rutas que tiene el live hoy	89
Rutas que tenemos nosotros	63
Rutas pendientes	26 (29 %)
Rutas nuevas aparecidas entre el 30-jul y el 9-sep	14 en 41 días
Ritmo de aparición	1 pantalla nueva cada 3 días

El dato importante no es el 29 % pendiente: es que **el hueco no se cierra**. Cada barrido encuentra más superficie nueva de la que hemos alcanzado a copiar desde el barrido anterior. El 30 de julio catalogamos 11 pantallas pendientes; seis semanas después son 26, y por el camino un módulo entero (ConceptOne) se ha reescrito con una carcasa distinta, un sistema de color propio y un modo oscuro que antes no existía.

Y la copia se degrada sola. La app de producción mueve cifras *en horas*: el 29 de julio, entre la mañana y la tarde del mismo día, una misma pantalla pasó de 9.631,60 € a 9.728,40 € de pendiente. Por eso nuestra regla interna es «el calco se fija a la foto». Estamos versionando fotografías de un sistema vivo.

3. Los cuatro costes reales de seguir así

a) La única fuente de verdad es un sistema que no controlamos

No hay especificación escrita de esta intranet en ningún sitio. La especificación es la pantalla del live. No existe documento de requisitos, ni esquema de datos, ni contrato de API que podamos consultar. Todo lo que sabemos lo sabemos porque lo hemos mirado.

b) El coste de sincronizar crece con el tamaño de la app

Copiar por captura tiene un coste proporcional a la superficie: cada pantalla nueva son horas de captura, medición, maquetado, datos inventados y tests. Con el repositorio original, incorporar una pantalla nueva es leer un componente que ya está escrito. La diferencia se agranda cada semana.

c) No tenemos datos, y no hay atajo para conseguirlos

Nuestra app enseña 7.966 líneas de datos falsos. Son suficientes para validar el diseño y para demostrar; no lo son para nada más. La arquitectura ya contempla el cambio —hay un `SupabaseAdapter` con sus 17 métodos y un interruptor por variable de entorno— pero **los 17 métodos lanzan una excepción**, porque no sabemos contra qué tablas hablarían. El enchufe está puesto; falta el otro extremo del cable.

d) La fidelidad tiene un techo que ya hemos tocado

Sin el código original medimos píxeles en lugar de leer la fuente. Eso produce errores que solo se descubren tarde: hemos tenido que corregir tonos de color de un chip, un título gris que habíamos teñido de color de marca, y una aprobación de cliente que resultó tener cuatro valores y no dos. Cada uno de esos hallazgos costó una captura nueva y una corrección. Con acceso al repositorio, ninguno habría llegado a existir.

4. Qué pasa exactamente si el live se cae

Escenario: mañana la app de producción deja de estar accesible —caída, cambio de credenciales, fin del acceso al usuario de prueba, o simplemente un rediseño que borra lo anterior.

Efecto inmediato: el boilerplate *sigue arrancando* —corre con datos locales—, pero pierde a la vez su especificación, su banco de datos y su método de verificación. Todo el trabajo pendiente se queda sin referencia contra la que hacerse.

Qué perderíamos, en concreto:

- **La especificación de las 26 pantallas que faltan.** No hay dónde consultarlas. Habría que inventarlas, y entonces ya no serían la intranet de Black Moose: serían otra cosa parecida.
- **La capacidad de verificar lo hecho.** Nuestro criterio de «correcto» es «coincide con el live». Sin live no hay criterio; las 413 capturas envejecen y no se pueden ampliar.
- **El modelo de datos.** Nunca lo hemos tenido. Lo poco que intuimos está deducido de cómo se ven las tablas por fuera.
- **La lógica de negocio que no se ve.** Cómo se calcula una comisión, cuándo se descuenta el stock, qué dispara una alerta. Todo eso está en el código del live y nosotros solo vemos el resultado.

Cuánto costaría recuperarlo. Reconstruir desde cero el modelo de datos y la lógica de 89 pantallas sin poder mirar el original es ingeniería inversa a ciegas. Tomando como base nuestro propio ritmo —dos meses de trabajo intensivo para llegar al 71 % de la *superficie visual*, sin nada de lógica ni de datos reales—, la estimación razonable para dejar el producto en pie de forma autónoma está en el orden de **varios meses de trabajo continuado**, y con un resultado que sería una aproximación, no la intranet. No es un riesgo teórico: es el estado en el que estamos hoy, todos los días, mientras el acceso dependa de que un sistema de terceros siga encendido.

5. Qué desbloquea el acceso al repositorio original

Con el código fuente delante, el trabajo deja de ser *adivinar cómo está hecho* y pasa a ser *decidir qué nos llevamos*.

- **Fidelidad exacta y gratis.** Los colores, los espaciados y los textos se leen del código. Se acaban las dobles medidas, el muestreo de PNG y las correcciones a posteriori.

- **Las 26 pantallas pendientes dejan de ser proyectos.** Pasan a ser lectura y adaptación. La estimación de ConceptOne v3 —seis fases, tres agentes— se reduce drásticamente cuando la fuente está disponible.
- **Se acaban los barridos.** Un `git diff` sustituye a la jornada de capturas y a los cinco barridos que llevamos. La detección de cambios pasa de días a segundos.
- **Recuperamos la lógica de negocio**, que es lo que de verdad da valor y lo único que una captura nunca enseña.

6. Qué desbloquea el acceso a Supabase

El live corre sobre Supabase —se ve en el propio token de sesión del navegador—. Nuestra arquitectura ya está preparada para hablar con él y hoy está vacía.

- **El esquema convierte 17 excepciones en 17 métodos.** Con las tablas y sus relaciones, el `SupabaseAdapter` se implementa; el interruptor `VITE_DATA_ADAPTER` ya existe y el cambio es de configuración, no de reescritura.
- **Se acaban las 7.966 líneas de datos inventados** y su mantenimiento.
- **Los 11 módulos que hoy leen literales** pueden pasar por el repositorio de datos como ya hacen los otros 5.
- **El boilerplate deja de ser una maqueta** y pasa a ser un cliente alternativo del mismo sistema: es la diferencia entre enseñar una pantalla y usarla.

7. Comparativa

TAREA	HOY (CALCO POR CAPTURA)	CON ACCESO
Detectar qué ha cambiado en el live	Barrido completo: ~1 jornada de agente, y no garantiza cobertura	<code>git diff</code> entre dos fechas: minutos
Incorporar una pantalla nueva	Capturar, medir, maquetar, inventar datos, testear	Leer el componente y adaptarlo
Fijar un color	Dos medidas independientes que deben coincidir	Leer la variable
Saber cómo se calcula una cifra	No se puede: solo vemos el resultado	Leer la función
Tener datos reales	Imposible	Conectar el adaptador que ya existe
Si el live se apaga	El trabajo pendiente se queda sin referencia	Seguimos: la fuente está en nuestras manos

8. Qué pedimos, y con qué garantías

La petición puede ser gradual. En orden de utilidad frente a coste de concesión:

#	PETICIÓN	ALCANCE MÍNIMO SUFICIENTE
---	----------	---------------------------

1	Esquema de la base de datos	Volcado de estructura sin datos (<code>pg_dump --schema-only</code>). No expone información personal y desbloquea el adaptador.
2	Lectura del repositorio original	Acceso de solo lectura, o un volcado del código en un momento dado. Con eso se acaban los barridos.
3	Acceso a Supabase en lectura	Usuario de solo lectura contra un entorno de <i>staging</i> o una réplica, con RLS activo. No hace falta producción.

Garantías por nuestra parte: acceso de solo lectura, nunca escritura; claves fuera del repositorio (`.env` ya está ignorado); datos personales fuera de los *seeds* y de las capturas; y el modo `mock` se mantiene como opción por defecto para desarrollo y tests, que seguirán corriendo sin tocar ningún sistema real.

9. Recomendación

Seguir con el calco por captura es viable a corto plazo y lo hemos demostrado: hay 89 pantallas catalogadas, 1.019 tests en verde y un módulo entero especificado esta misma semana. Pero es un método cuyo coste crece con el tamaño del objetivo y cuya continuidad depende de un sistema ajeno que siga encendido y accesible.

La recomendación es pedir, en este orden: el esquema de la base de datos, la lectura del repositorio y un acceso de lectura a Supabase. El primero es el que más desbloquea con el menor coste de concesión, porque no expone datos y convierte un adaptador vacío en una aplicación real. Y conviene pedirlo ahora, mientras el live sigue accesible y podemos contrastar lo que recibamos con lo que ya tenemos medido: hacerlo después de una caída sería empezar la casa por el tejado.

Elaborado el 9 de septiembre de 2026 a partir de: barrido del live del mismo día (89 rutas inventariadas), informe de deltas del 30 de julio de 2026, y métricas del repositorio `Boilerplate` en `main` (`990dd7e`). Todas las cifras del apartado 1 y 2 son medidas, no estimaciones; la estimación temporal del apartado 4 se indica como tal.